Veeva Java MCQ and SQL Revision

This sheet contains important Java output-based MCQs and 7 SQL query patterns for Veeva preparation.

Java MCQs

1. String Immutability

What is the output?

```
public class Test {
    public static void main(String[] args) {
        String s = "abc";
        s.toUpperCase();
        System.out.println(s);
    }
}
```

A. ABC

B. abc

C. Compile-time error

D. Runtime error

Answer: B

Explanation: String is immutable. `toUpperCase()` returns a new string, but it is not assigned back to `s`.

2. Method Overloading With null

What is the output?

```
public class Code {
    public static void main(String[] args) {
        method(null);
    }

    public static void method(Object o) {
        System.out.println("Object method");
    }

    public static void method(String s) {
        System.out.println("String method");
    }
}
```

A. Object method

B. String method

C. Compile-time error

D. Runtime error

Answer: B

Explanation: `String` is more specific than `Object`, so `method(String s)` is selected.

3. String Concatenation

What is the output?

```
public class Test {
    public static void main(String[] args) {
        String s = new String("5");
        System.out.println(1 + 10 + s + 1 + 10);
    }
}
```

A. 115110

B. 111510

C. 27

D. 151110

Answer: A

Explanation: `1 + 10` is evaluated first as 11. After string appears, remaining + operations become concatenation.

4. Static Block, Instance Block, Constructor

What is the output?

```
class A {
    static {
        System.out.print("1");
    }

    {
        System.out.print("2");
    }

    public A() {
        System.out.print("3");
    }
}

public class Test {
    public static void main(String[] args) {
        A a1 = new A();
        A a2 = new A();
    }
}
```

A. 12323

B. 121323

C. 2323

D. 112233

Answer: A

Explanation: Static block runs once. For each object, instance block runs before constructor.

5. Instance Method Overloading With null

What is the output?

```
public class Test {
    public static void main(String[] args) {
        new Test().print(null);
    }

    public void print(Object o) {
        System.out.println("Object");
    }

    public void print(String s) {
        System.out.println("String");
    }
}
```

A. Object

B. String

C. Compile-time error

D. Runtime error

Answer: B

Explanation: `String` is more specific than `Object`.

6. Floating Point Precision

What is the output?

```
public class Test {
    public static void main(String[] args) {
        double a = 0.1 + 0.2;
        System.out.println(a == 0.3);
        System.out.println(a);
    }
}
```

A. true and 0.3

B. false and 0.30000000000000004

C. true and 0.30000000000000004

D. Compile-time error

Answer: B

Explanation: Decimal floating-point values are not always represented exactly in binary.

7. try-catch-finally

What is the output?

```
public class Test {
    public static void main(String[] args) {
        try {
            System.out.println("A");
            throw new RuntimeException("Test");
        } catch (RuntimeException e) {
            System.out.println("B");
        } finally {
            System.out.println("C");
        }
    }
}
```

A. A

B. A B

C. A B C

D. A C

Answer: C

Explanation: The exception is caught, and `finally` always executes.

8. Varargs Overloading

What is the output?

```
public class Test {
    public static void main(String[] args) {
        new Test().print(1, 2, 3);
    }

    public void print(int... numbers) {
        System.out.println("int...");
    }

    public void print(Integer i1, Integer i2) {
        System.out.println("Integer");
    }
}
```

A. int...

B. Integer

C. Compile-time error

D. Runtime error

Answer: A

Explanation: The call has 3 arguments. `print(Integer, Integer)` accepts only 2 arguments, so varargs is selected.

9. PriorityQueue

What is the output?

```
import java.util.*;

public class priorityQueue {
    public static void main(String[] args) {
        PriorityQueue<Integer> queue = new PriorityQueue<>();
        queue.add(11);
        queue.add(10);
        queue.add(22);
        queue.add(5);
        queue.add(12);
        queue.add(2);

        while (queue.isEmpty() == false)
            System.out.printf("%d ", queue.remove());

        System.out.println();
    }
}
```

A. 11 10 22 5 12 2

B. 2 12 5 22 10 11

C. 2 5 10 11 12 22

D. 22 12 11 10 5 2

Answer: C

Explanation: Default `PriorityQueue<Integer>` is a min-heap, so removal gives ascending order.

10. TreeSet

What is the output?

```
import java.util.*;

public class Treeset {
    public static void main(String[] args) {
        TreeSet<String> treeSet = new TreeSet<>();

        treeSet.add("Geeks");
        treeSet.add("For");
        treeSet.add("Geeks");
        treeSet.add("GeeksforGeeks");

        for (String temp : treeSet)
            System.out.printf(temp + " ");

        System.out.println();
    }
}
```

A. Geeks For Geeks GeeksforGeeks

B. Geeks For GeeksforGeeks

C. For Geeks GeeksforGeeks

D. For GeeksforGeeks Geeks

Answer: C

Explanation: `TreeSet` removes duplicates and stores elements in sorted order.

11. LinkedList removeAll

What is the output?

```
import java.util.*;

public class linkedList {
    public static void main(String[] args) {
        List<String> list1 = new LinkedList<>();
        list1.add("Geeks");
        list1.add("For");
```

```
        list1.add("Geeks");
        list1.add("GFG");
        list1.add("GeeksforGeeks");

        List<String> list2 = new LinkedList<>();
        list2.add("Geeks");

        list1.removeAll(list2);

        for (String temp : list1)
            System.out.printf(temp + " ");

        System.out.println();
    }
}
```

A. For Geeks GFG GeeksforGeeks

B. For GeeksforGeeks GFG

C. For GFG for

D. For GFG GeeksforGeeks

Answer: D

Explanation: `removeAll` removes all occurrences of `"Geeks"` from `list1`.

12. Iterator Type Mismatch

What is the output?

```
import java.util.*;

public class stack {
    public static void main(String[] args) {
        List<String> list = new LinkedList<>();
        list.add("Geeks");
        list.add("For");
        list.add("Geeks");
        list.add("GeeksforGeeks");
        Iterator<Integer> iter = list.iterator();

        while (iter.hasNext())
            System.out.printf(iter.next() + " ");

        System.out.println();
    }
}
```

A. Geeks For Geeks GeeksforGeeks

B. GeeksforGeeks Geeks For Geeks

C. Runtime Error

D. Compilation Error

Answer: D

Explanation: `list.iterator()` returns `Iterator<String>`, not `Iterator<Integer>`.

13. Private Constructor And Private Field

What is the output?

```
class Helper {
    private int data;

    private Helper() {
        data = 5;
    }
}

public class Test {
    public static void main(String[] args) {
        Helper help = new Helper();
        System.out.println(help.data);
    }
}
```

A. Compilation error

B. 5

C. Runtime error

D. None of these

Answer: A

Explanation: The constructor is private and `data` is private, so both accesses fail outside `Helper`.

14. Runnable Constructor Syntax

What is the output?

```
public class Test implements Runnable {
    public void run() {
        System.out.printf(" Thread's running ");
    }

    try {
        public Test() {
            Thread.sleep(5000);
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    public static void main(String[] args) {
        Test obj = new Test();
        Thread thread = new Thread(obj);
        thread.start();
        System.out.printf(" GFG ");
    }
}
```

A. GFG Thread's running

B. Thread's running GFG

C. Compilation error

D. Runtime error

Answer: C

Explanation: A constructor cannot be declared inside a `try` block.

7 SQL Query Patterns

1. 2nd Highest Salary In Engineering Department

If more than one person shares the highest salary, select the next distinct salary.

```
SELECT MAX(e.salary) AS salary
FROM employees e
JOIN departments d
ON e.department_id = d.id
WHERE d.name = 'engineering'
AND e.salary < (
    SELECT MAX(e2.salary)
    FROM employees e2
    JOIN departments d2
    ON e2.department_id = d2.id
    WHERE d2.name = 'engineering'
);
```

Pattern: `MAX(salary)` less than the highest salary.

2. Three-Day Rolling Average For Deposits By Day

```
WITH daily_deposits AS (
    SELECT
        DATE_FORMAT(created_at, '%Y-%m-%d') AS dt,
        SUM(transaction_value) AS daily_total
    FROM bank_transactions
```

```
    WHERE transaction_value > 0
    GROUP BY DATE_FORMAT(created_at, '%Y-%m-%d')
)
SELECT
    dt,
    AVG(daily_total) OVER (
        ORDER BY dt
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS rolling_three_day
FROM daily_deposits
ORDER BY dt;
```

Pattern: daily aggregation first, then window function.

3. Customers With More Than 3 Transactions In Both 2019 And 2020

```
SELECT u.name AS customer_name
FROM users u
JOIN transactions t
ON u.id = t.user_id
WHERE YEAR(t.created_at) IN (2019, 2020)
GROUP BY u.id, u.name
HAVING SUM(CASE WHEN YEAR(t.created_at) = 2019 THEN 1 ELSE 0 END) > 3
AND SUM(CASE WHEN YEAR(t.created_at) = 2020 THEN 1 ELSE 0 END) > 3;
```

Pattern: conditional count using `SUM(CASE WHEN ...)`.

4. Histogram Of Comments Per User In January 2020

Users with no January comments must be counted in the `0` bucket.

```
WITH user_comment_counts AS (
    SELECT
        u.id,
        COUNT(c.user_id) AS comment_count
    FROM users u
    LEFT JOIN comments c
    ON u.id = c.user_id
    AND c.created_at >= '2020-01-01'
    AND c.created_at < '2020-02-01'
    GROUP BY u.id
)
SELECT
    comment_count,
    COUNT(*) AS frequency
FROM user_comment_counts
GROUP BY comment_count
ORDER BY comment_count;
```

Pattern: count per user first, then group by the count.

5. Top 3 Departments By Percentage Of Employees Over 100K

Only include departments with at least 10 employees.

```
SELECT
    d.name AS department_name,
    COUNT(e.id) AS number_of_employees,
    AVG(CASE WHEN e.salary > 100000 THEN 1.0 ELSE 0.0 END) AS percentage_over_100k
FROM departments d
JOIN employees e
ON d.id = e.department_id
GROUP BY d.id, d.name
HAVING COUNT(e.id) >= 10
ORDER BY percentage_over_100k DESC
LIMIT 3;
```

Pattern: percentage = average of 1s and 0s.

6. Employees Above Average Salary

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Pattern: subquery in `WHERE`.

7. Third Transaction Per User

```
WITH ranked_transactions AS (
    SELECT
        user_id,
        spend,
        transaction_date,
        ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY transaction_date ASC
        ) AS rn
    FROM transactions
)
SELECT user_id, spend, transaction_date
FROM ranked_transactions
WHERE rn = 3;
```

Pattern: `ROW_NUMBER()` with `PARTITION BY`.

SQL Memory Hooks

- Second highest distinct salary: `MAX(salary) WHERE salary < MAX(salary)`
- Rolling average: `AVG(...) OVER (ORDER BY date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
- Conditional count: `SUM(CASE WHEN condition THEN 1 ELSE 0 END)`
- Histogram: count per entity first, then group by that count
- Percentage: `AVG(CASE WHEN condition THEN 1.0 ELSE 0.0 END)`
- Ranking: `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`